

E3 — Escenarios de Prueba

Sistema de Tickets e Incidentes — Entrega #3 · Escenarios de Prueba

Universidad Galileo · Postgrado en Diseño y Desarrollo de Software · Infraestructura en la Nube Ciclo Mayo–Junio 2026

Equipo:

- Luis André Morales
 - Erick Estuardo Saban
-

0. Qué cambió desde la entrega anterior

Esta es la **primera versión del documento de Escenarios de Prueba** y nace como artefacto separado de `docs/E2_PlanDePruebas.md`. Toma como insumo la Entrega #2 (el plan de pruebas con los casos `TU-`, `TI-`, `TE2E-`, `TUAT-`, `TSEC-`, `TC-`, `TS-`, `TUX-`) y la Entrega #1 (los 8 casos de uso CU-01 a CU-08 y sus criterios de éxito).

La diferencia frente a E2 es de **formato y propósito**:

- E2 documenta la **estrategia** del plan de pruebas (niveles, ambientes, herramientas, criterios de entrada/salida, riesgos, cronograma) y enumera casos representativos por tipo en tablas resumidas con columnas `Descripción / Precondición / Pasos / Resultado / Prioridad`.
- E3 (este documento) produce **escenarios ejecutables** siguiendo de manera estricta los **ocho campos prescritos por la rúbrica del curso**:
 1. **Título descriptivo**
 2. **Pasos a ejecutar**
 3. **Al menos 1 resultado esperado**
 4. **Estado de ejecución**
 5. **Precondiciones**
 6. **Acciones post ejecución**
 7. **Data**
 8. **Adjuntos**

Para evidenciar el uso de una **herramienta de gestión de pruebas con capa gratuita permanente** — como pide la rúbrica — se eligió **Qase.io** después de comparar contra TestRail (descartado: solo trial de 14–

30 días), TestLink (self-hosted, fricción innecesaria), GitHub Issues (no es test-management) y Notion (no especializado). La justificación detallada vive en §3.

El alcance se acotó deliberadamente a **24 escenarios funcionales — tres por cada uno de los 8 CUs**: uno de camino feliz, uno de error controlado o RBAC denegado, y uno de borde / concurrencia / asíncrono. Las pruebas no funcionales (carga, estrés, seguridad invasiva, UX heurística) **no se duplican aquí** porque ya viven completas en E2 §8–§11; este documento es la capa funcional ejecutable.

Ningún archivo previo se modifica: `E1_TicketSystem.md` , `E2_PlanDePruebas.md` y `E2_PlanDePruebas.pdf` quedan inmutables como entregas históricas.

1. Propósito y alcance

1.1 Propósito

1. Proveer un **catálogo ejecutable de escenarios funcionales** que permita al equipo (y a un evaluador externo) **ejecutar manualmente o automatizar** la verificación de cada Caso de Uso CU-01 a CU-08.
2. Demostrar el uso de una **herramienta de gestión de pruebas** (Qase.io) con su capa gratuita, mostrando cómo los escenarios se modelan, agrupan en suites por CU y se registran en runs de ejecución.
3. Mantener **trazabilidad bidireccional** con el plan de pruebas E2: cada escenario `EP-` referencia los IDs `TI-` , `TUAT-` o `TE2E-` correspondientes de E2.
4. Servir de **base de regresión** para entregas futuras (D2, D3, D4 del cronograma de E2 §15): cada escenario se vuelve a ejecutar cuando un cambio de código toca el CU asociado.

1.2 Alcance — qué entra en este documento

Categoría	Incluido	Referencia
Escenarios funcionales por CU	24 escenarios (3 × 8 CUs)	E1 §3 (CU-01..CU-08); E2 §4 a §7
Trazabilidad cruzada a E2	Cada escenario referencia uno o más casos <code>TI-</code> / <code>TUAT-</code> / <code>TE2E-</code> / <code>TU-</code>	E2 §12 (matriz de trazabilidad)
Registro en herramienta de gestión	Mapeo a Qase.io + CSV importable (<code>docs/E3_qase_import.csv</code>)	§3 de este documento
Mockups como evidencia visual	Referenciados en el campo "Adjuntos" cuando aplica	<code>docs/mockups/01..08</code>

1.3 Fuera del alcance de este documento

- **Pruebas no funcionales** (carga, estrés, seguridad invasiva con ZAP/sqlmap, UX heurística): siguen viviendo en E2 §8–§11 sin duplicación aquí.

- **Pruebas unitarias** puras (TU- de E2 §4): se referencian como apoyo cuando un EP- cubre la misma lógica de dominio, pero los casos TU- no se reescriben con el formato de 8 campos (no aplica el campo "Data" ni "Adjuntos" en pruebas aisladas de función).
- **Pruebas de infraestructura como código** (terraform plan , tflint , tfsec , checkov): siguen viviendo en E2 §3.7 y `.github/workflows/terraform-ci.yml`.
- **Diseño de la suite automatizada** (selección de runner, paralelización, integración con Qase API): se pospone a la Entrega #4 cuando exista código de aplicación.

2.1 Nomenclatura de IDs

Los términos siguen el glosario consolidado en E1 §3 y E2 §2; aquí solo se resume lo estrictamente necesario:

Término	Definición operativa
Reportante	Rol que abre tickets y consulta los propios; sin permiso de asignación ni cambio de estado.
Agente	Rol que toma tickets de la cola, cambia estado, resuelve y reasigna.
Administrador	Rol con permisos plenos, incluye descarga de reportes y configuración de SLA.
Severidad	Crítica / Alta / Media / Baja (input del reportante).
Prioridad	Número calculado a partir de severidad + tipo (Incidente > Solicitud).
SLA	Tiempo máximo de actividad antes del escalamiento automático L1→L2→L3.
TKT-XXXX	Formato del identificador público del ticket (autoincremental con padding).
ENT-001..ENT-005	IDs de ambiente definidos en E2 §2.2 (local/dev , CI ephemeral , dev shared , staging , prod).

2.4 Trazabilidad cruzada con E2

Cada escenario incluye una fila **"Relacionado con (E2)"** que enumera los IDs de E2 que cubren el mismo comportamiento. Esta fila permite:

- Saltar desde Qase.io al detalle técnico del caso en E2 (por ejemplo, ver la herramienta exacta o el dataset usado).
- Justificar el "Definition of Done" del nivel correspondiente en E2 §2.4 (todos los EP- aprobados ⇒ los TI- / TUAT- asociados se consideran verificados a nivel de aceptación).
- Evitar duplicación: si un comportamiento ya está cubierto por un TI- automatizado, el EP- solo añade la lente del usuario final (input/output observable).

3. Herramienta de gestión: Qase.io

3.1 Justificación de la elección

Se evaluaron cuatro alternativas contra cuatro criterios obligatorios derivados de la restricción del usuario y del slide del profesor:

Criterio	Peso
C1 — Capa gratuita permanente (sin caducidad de trial)	Bloqueante
C2 — Cubre los 8 campos del slide nativamente o vía custom fields	Bloqueante
C3 — Permite exportar los escenarios a PDF / CSV para anexar a la entrega	Alto
C4 — Integración o referencia desde el repo GitHub del proyecto	Medio

Herramienta	C1 Capa gratuita	C2 Campos	C3 Export	C4 GitHub	Veredicto
Qase.io	✅ Free permanente (3 usuarios, ~200 cases, runs/mes)	✅ Nativo: title, preconditions, steps, expected result, post-conditions, custom fields para data y attachments	✅ PDF + CSV + XLSX	✅ App GitHub oficial, integración con Issues	Elegida
TestRail	❌ Solo trial 14–30 días; después \$37/usuario/mes	✅	✅	Parcial (add-ons)	Descartada por C1 — el trial vence durante el ciclo
TestLink	✅ Open source, self-hosted	✅	Parcial (HTML/XML)	❌	Descartada — exige montar PHP+MySQL local, fricción innecesaria para entrega académica
GitHub Issues + template	✅ Gratis con el repo	Parcial (vía template, sin campos tipados)	Manual (ningún export estructurado)	✅ Nativo	Descartada por C2/C3 — no es test-management
Notion	✅ Free tier amplio	✅ Con base de datos custom	✅ PDF	Parcial	Descartada — no es purpose-built para test management; complica reporting

Resultado: Qase.io es la única herramienta que satisface los cuatro criterios sin caducidad ni montaje de infraestructura.

3.2 Configuración del proyecto en Qase

Aspecto	Decisión
Nombre del proyecto	Sistema de Tickets – E3 Escenarios Funcionales
Plan	Free
Estructura de suites	Una suite por CU: CU-01 Abrir ticket , CU-02 Clasificar y asignar , ..., CU-08 Solicitud de servicio
Mapeo de campos del slide a Qase	Título → Title ; Pasos → Steps (array); Resultado esperado → Expected result por paso; Estado de ejecución → Status del run; Precondiciones → Pre-conditions ; Acciones post ejecución → Post-conditions ; Data → custom field Test data (multiline); Adjuntos → Attachments (upload)
Severity / Priority	Se reusa Severity y Priority nativos de Qase con los valores usados en el documento
Tags	cu-01 .. cu-08 , feliz / negativo / borde , rbac cuando aplica
Integración GitHub	App de Qase instalada en el repo ticket-system-infra , sin permisos de escritura; solo para enlazar issues desde cada escenario

3.3 Cómo replicar los escenarios en Qase

1. Crear cuenta gratuita en <https://qase.io> (free plan).
2. Crear el proyecto Sistema de Tickets – E3 Escenarios Funcionales .
3. Crear las 8 suites (CU-01 a CU-08).
4. Ir a Settings → Import → CSV e importar docs/E3_qase_import.csv (entregable de esta misma fase) con el mapeo de columnas:
 - id → External ID
 - suite → Suite
 - title → Title
 - preconditions , steps , expected_result , post_actions → campos homónimos
 - data , attachments , related_e2 → custom fields
 - priority , severity → campos nativos
5. Validar visualmente que las 24 entradas quedaron repartidas 3 por suite.
6. Crear un primer Test Run llamado Baseline – Diseño con todos los escenarios en estado No ejecutado (estado por defecto al importar).

3.4 Acceso y evidencia para el profesor

- El proyecto puede compartirse en modo **lectura pública** (Qase free permite enlace público de run): la URL se incluirá en la versión final del PDF cuando se cree la cuenta.
- Como respaldo offline (por si se pierde la cuenta o se excede el free tier), el repositorio versiona el CSV (docs/E3_qase_import.csv), que puede reimportarse en cualquier momento.

4. Escenarios por Caso de Uso

Nota de lectura: los 24 escenarios siguen la misma estructura de tabla de dos columnas (Campo / Valor) con las 8 etiquetas del slide en orden fijo, más tres filas auxiliares al final (Tipo, Prioridad, Relacionado con E2). Esta forma vertical es la única que mantiene legibilidad dado que el campo "Pasos a ejecutar" es multi-línea.

4.1 CU-01 — Abrir un ticket de incidente

EP-CU01-01 — Reportante crea incidente crítico con adjunto

Campo	Valor
Título descriptivo	Reportante autenticado crea un incidente crítico con un adjunto PNG y recibe ticket TKT-XXXX en estado Abierto .
Precondiciones	1. Ambiente ENT-004 staging operativo (API, Cognito, RDS, S3, SQS). 2. Usuario reportante1@galileo.gt existe en Cognito con rol Reportante . 3. Bucket S3 tickets-attachments-staging accesible. 4. URL prefirmada de subida solicitada al endpoint POST /attachments/presign y archivo captura.png (300 KB) subido exitosamente con la URL.
Data	tipo=Incidente , severidad=Crítica , titulo="API caída en prod" , descripcion="HTTP 503 en /v1/orders desde 14:32 UTC" , attachmentKeys=["tickets/2026/05/captura.png"] . JWT del reportante en Authorization: Bearer
Pasos a ejecutar	1. POST /auth/login con credenciales del reportante; obtener JWT. 2. POST /tickets con el JSON del campo Data. 3. Capturar el id devuelto. 4. GET /tickets/{id} con el mismo JWT. 5. GET /tickets/{id}/history .
Resultado esperado	• Paso 2 → 201 Created . • Body con id formato TKT-XXXX , estado="Abierto" , prioridad numérica calculada (máxima entre las 4 severidades para Incidente) , createdAt con sufijo Z (UTC). • Paso 4 → 200 con el ticket y attachmentKeys poblado. • Paso 5 → un único evento TICKET_CREATED con autor=reportante1@galileo.gt y timestamp UTC.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Anotar el TKT-XXXX generado en el log del run para usarlo en escenarios encadenados. 2. Programar limpieza del adjunto en S3 mediante el próximo re-seed nocturno de staging (no eliminar manualmente para no romper el escenario siguiente que consulta el historial). 3. Registrar el resultado del run en Qase.io.
Adjuntos	docs/mockups/03_crear_incidente.png (mockup del formulario); evidencias/EP-CU01-01-response.json (capturar al ejecutar).
Tipo	Camino feliz
Prioridad	Crítica
Relacionado con (E2)	TI-001 , TI-006 , TI-009 , TUAT-001 , TE2E-001

EP-CU01-02 — Crear ticket sin título devuelve 400

Campo	Valor
Título descriptivo	El API rechaza con <code>400 Bad Request</code> un intento de crear ticket sin el campo obligatorio <code>título</code> y detalla el campo faltante.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. Reportante autenticado con JWT válido.
Data	Cuerpo JSON: <code>{ "tipo": "Incidente", "severidad": "Alta", "descripcion": "Pruebas de validación" }</code> — sin <code>título</code> .
Pasos a ejecutar	1. Obtener JWT del reportante. 2. <code>POST /tickets</code> con el cuerpo del campo Data. 3. Leer el código de respuesta y el body.
Resultado esperado	• Respuesta <code>400 Bad Request</code>. • Body con código de error <code>VALIDATION_ERROR</code> y detalle <code>{ "field": "título", "reason": "REQUIRED" }</code>. • Ningún registro insertado en la tabla <code>tickets</code> (verificable consultando <code>GET /tickets</code> con filtros).
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Verificar que no quedó ningún ticket huérfano en RDS (consulta SQL si hace falta). 2. Registrar el resultado en Qase.io.
Adjuntos	<code>docs/mockups/03_crear_incidente.png</code> (formulario donde el campo <code>título</code> es obligatorio).
Tipo	Error controlado
Prioridad	Alta
Relacionado con (E2)	<code>TI-003</code> , <code>TU-013</code>

EP-CU01-03 — Idempotencia: doble POST con misma Idempotency-Key

Campo	Valor
Título descriptivo	Dos <code>POST /tickets</code> con el mismo header <code>Idempotency-Key</code> crean un único ticket; el segundo POST devuelve <code>200 OK</code> con el mismo <code>TKT-XXXX</code> .
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. Reportante autenticado. 3. Llave de idempotencia generada por el cliente: <code>UUID v4 e3-cu01-03-<timestamp></code> .
Data	Cuerpo idéntico en ambos POST: <code>{ "tipo": "Incidente", "severidad": "Media", "titulo": "Idempotencia E3", "descripcion": "Verificación de Idempotency-Key" }</code> . Header <code>Idempotency-Key: e3-cu01-03-<timestamp></code> .
Pasos a ejecutar	1. Primer <code>POST /tickets</code> con el header <code>Idempotency-Key</code> . 2. Capturar el <code>id</code> devuelto (debe ser <code>201 Created</code>). 3. Segundo <code>POST /tickets</code> con exactamente el mismo cuerpo y el mismo header. 4. Comparar <code>id</code> y código de respuesta. 5. <code>GET /tickets/{id}/history</code> para confirmar un único evento de creación.
Resultado esperado	• Paso 1 → <code>201 Created</code> con <code>id=TKT-N</code> . • Paso 3 → <code>200 OK</code> con <code>id=TKT-N</code> (mismo identificador, sin crear ticket nuevo). • Paso 5 → exactamente un evento <code>TICKET_CREATED</code> . • Conteo total de tickets del reportante: aumenta solo en 1.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Limpiar el ticket en el próximo re-seed. 2. Documentar el <code>Idempotency-Key</code> usado en el log del run (no reusar en el próximo run para no contaminar). 3. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Borde / concurrencia
Prioridad	Alta
Relacionado con (E2)	<code>TI-007</code>

4.2 CU-02 — Clasificar y asignar

EP-CU02-01 — Agente toma ticket de la cola priorizada

Campo	Valor
Título descriptivo	Un agente autenticado consulta la cola ordenada por prioridad y se autoasigna el primer ticket crítico; el reportante recibe notificación.
Precondiciones	1. Ambiente ENT-004 staging . 2. Existe TKT-0042 Incidente Crítico en estado Abierto , sin assignee. 3. Existen al menos 3 tickets de menor prioridad ya Abiertos para validar el orden. 4. Usuario agente1@galileo.gt con rol Agente . 5. Cola SQS notifications-queue vacía o monitoreada.
Data	Header Authorization: Bearer <JWT-agente1> . Cuerpo del assign: { "assigneeId": "agente1@galileo.gt" } .
Pasos a ejecutar	1. Login del agente; obtener JWT. 2. GET /tickets?orderBy=prioridad&direction=desc&limit=10 . 3. Verificar que TKT-0042 aparece en primera posición. 4. POST /tickets/TKT-0042/assign con el cuerpo Data. 5. GET /tickets/TKT-0042 . 6. Consumir la cola SQS de notificaciones (o revisar Mailpit) durante 30 s.
Resultado esperado	• Paso 2 → 200 OK con TKT-0042 en posición 1. • Paso 4 → 200 OK ; ticket pasa a EnProgreso ; assignee="agente1@galileo.gt" . • Paso 5 → confirma el estado y assignee. • Paso 6 → un mensaje en la cola con eventType=TICKET_ASSIGNED , destinatario=reportante original, body contiene el nombre del agente.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Revertir el ticket a Abierto y desasignar para permitir re-ejecución (o usar otro TKT- en próximo run). 2. Drenar la cola SQS si quedó algún mensaje no procesado del run. 3. Registrar en Qase.io.
Adjuntos	docs/mockups/02_cola_tickets.png (cola priorizada que muestra el orden por severidad).
Tipo	Camino feliz
Prioridad	Crítica
Relacionado con (E2)	TI-040 , TUAT-002 , TE2E-002

EP-CU02-02 — Reportante NO puede asignar (RBAC denegado)

Campo	Valor
Título descriptivo	Un reportante autenticado recibe <code>403 Forbidden</code> al intentar autoasignarse un ticket; ningún cambio se persiste.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. <code>TKT-0050</code> Incidente Alta, estado <code>Abierto</code> , sin assignee. 3. Usuario <code>reportante2@galileo.gt</code> con rol <code>Reportante</code> .
Data	JWT del reportante. Cuerpo: <code>{ "assigneeId": "reportante2@galileo.gt" }</code> .
Pasos a ejecutar	1. Login del reportante; obtener JWT. 2. <code>POST /tickets/TKT-0050/assign</code> con el cuerpo Data. 3. Leer código y body. 4. <code>GET /tickets/TKT-0050</code> para confirmar que sigue sin assignee.
Resultado esperado	• Paso 2 → <code>403 Forbidden</code> . • Body contiene código <code>ROLE_NOT_ALLOWED</code> y razón legible (<code>role=Reportante</code> , <code>action=assign</code>). • Paso 4 → <code>assignee=null</code> , <code>estado=Abierto</code> (sin cambios). • Ningún evento en <code>ticket_events</code> .
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Confirmar en CloudWatch que se loguea el intento denegado con <code>userId</code> , <code>ticketId</code> y motivo. 2. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Error controlado / RBAC
Prioridad	Crítica
Relacionado con (E2)	<code>TI-024</code> , <code>TU-007</code> , <code>TSEC-001</code> (RBAC matrix)

EP-CU02-03 — Reasignación A→B notifica a ambos agentes

Campo	Valor
Título descriptivo	Reasignar un ticket en <code>EnProgreso</code> de un agente A a un agente B produce dos notificaciones distintas en la cola SQS y actualiza el historial.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. <code>TKT-0060</code> Incidente Alta, estado <code>EnProgreso</code> , <code>assignee=agenteA@galileo.gt</code> . 3. Usuario <code>agenteB@galileo.gt</code> con rol <code>Agente</code> . 4. Usuario <code>admin1@galileo.gt</code> con rol <code>Administrador</code> (el reasignador).
Data	JWT del admin. Cuerpo del reassign: <code>{ "assigneeId": "agenteB@galileo.gt", "reason": "Rotación de turno" }</code> .
Pasos a ejecutar	1. Login del admin. 2. <code>POST /tickets/TKT-0060/assign</code> con el cuerpo Data. 3. <code>GET /tickets/TKT-0060/history</code> y verificar el último evento. 4. Consumir la cola SQS durante 30 s. 5. Verificar bandejas (Mailpit) de A y B.
Resultado esperado	• Paso 2 → <code>200 OK</code>; <code>assignee=agenteB@galileo.gt</code>. • Paso 3 → último evento <code>TICKET_REASSIGNED</code> con <code>from=A</code>, <code>to=B</code>, <code>reason="Rotación de turno"</code>, <code>actor=admin1</code>. • Paso 4 → dos mensajes en la cola, uno por agente. • Paso 5 → ambos agentes recibieron email distinto (A: "ya no eres responsable de TKT-0060"; B: "se te asignó TKT-0060").
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Drenar Mailpit del entorno. 2. Reasignar el ticket de vuelta a A para permitir re-ejecución. 3. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Borde
Prioridad	Media
Relacionado con (E2)	<code>TI-043</code>

4.3 CU-03 — Resolver con causa raíz y solución

EP-CU03-01 — Agente resuelve ticket con causa raíz y solución

Campo	Valor
Título descriptivo	El agente cierra un ticket en <code>EnProgreso</code> con causa raíz y solución aplicada; el reportante recibe notificación de resolución.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. <code>TKT-0070</code> Incidente Crítico, estado <code>EnProgreso</code> , <code>assignee=agente1</code> . 3. Agente autenticado. 4. Cola SQS de notificaciones monitoreada.
Data	JWT de agente1. Cuerpo: <code>{ "estado": "Resuelto", "causaRaiz": "Memoria saturada en pod orders-api", "solucionAplicada": "Reinicio + ajuste de limits a 1Gi", "tiempoResolucionMin": 47 }</code> .
Pasos a ejecutar	1. <code>PATCH /tickets/TKT-0070/state</code> con el cuerpo Data. 2. <code>GET /tickets/TKT-0070</code> . 3. <code>GET /tickets/TKT-0070/history</code> . 4. Verificar Mailpit para email al reportante original.
Resultado esperado	• Paso 1 → <code>200 OK</code> ; ticket en estado <code>Resuelto</code> ; campos <code>causaRaiz</code> y <code>solucionAplicada</code> persistidos. • Paso 3 → último evento <code>RESOLVED</code> con autor, timestamp UTC y ambos campos. • Paso 4 → email al reportante con subject <code>[TKT-0070] Resuelto</code> y el contenido de la solución.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Si se quiere re-ejecutar, crear un nuevo ticket en <code>EnProgreso</code> (no revertir el <code>Resuelto</code> , porque la transición inversa es ilegal por diseño). 2. Drenar Mailpit. 3. Registrar en Qase.io.
Adjuntos	<code>docs/mockups/05_detalle_resolucion.png</code> .
Tipo	Camino feliz
Prioridad	Crítica
Relacionado con (E2)	<code>TI-022</code> , <code>TUAT-003</code> , <code>TE2E-003</code> , <code>TU-016</code>

EP-CU03-02 — Resolución sin causa raíz devuelve 422

Campo	Valor
Título descriptivo	Intentar pasar un ticket a Resuelto sin causaRaiz ni solucionAplicada devuelve 422 Unprocessable Entity ; el ticket permanece en EnProgreso .
Precondiciones	1. Ambiente ENT-004 staging . 2. TKT-0071 Incidente Media en EnProgreso , asignado a agente1 .
Data	JWT de agente1. Cuerpo: { "estado": "Resuelto" } (sin causaRaiz ni solucionAplicada).
Pasos a ejecutar	1. PATCH /tickets/TKT-0071/state con el cuerpo Data. 2. Leer código y body. 3. GET /tickets/TKT-0071 para confirmar el estado.
Resultado esperado	• Paso 1 → 422 Unprocessable Entity . • Body con código MISSING_RESOLUTION_FIELDS y la lista ["causaRaiz", "solucionAplicada"] . • Paso 3 → ticket sigue en EnProgreso . • Ningún evento RESOLVED agregado al historial.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io. 2. Dejar el ticket en EnProgreso (sirve de seed para re-ejecución).
Adjuntos	docs/mockups/05_detalle_resolucion.png (campos obligatorios marcados con asterisco).
Tipo	Error controlado
Prioridad	Crítica
Relacionado con (E2)	TI-021 , TU-016

EP-CU03-03 — Transición ilegal Abierto→Resuelto

Campo	Valor
Título descriptivo	Un PATCH de estado intentando saltar directamente de Abierto a Resuelto (sin pasar por EnProgreso) devuelve 409 Conflict con código ILLEGAL_STATE_TRANSITION.
Precondiciones	1. Ambiente ENT-004 staging. 2. TKT-0072 Incidente Alta, estado Abierto, asignado a agente1.
Data	JWT de agente1. Cuerpo: { "estado": "Resuelto", "causaRaiz": "X", "solucionAplicada": "Y" }.
Pasos a ejecutar	1. PATCH /tickets/TKT-0072/state con el cuerpo Data. 2. Leer código y body. 3. GET /tickets/TKT-0072 para confirmar que sigue en Abierto.
Resultado esperado	• Paso 1 → 409 Conflict. • Body con código ILLEGAL_STATE_TRANSITION, from=Abierto, to=Resuelto, y allowedTransitions=["EnProgreso"]. • Paso 3 → ticket sigue en Abierto.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io. 2. Dejar el ticket en Abierto para re-ejecución.
Adjuntos	(sin adjuntos)
Tipo	Borde
Prioridad	Alta
Relacionado con (E2)	TI-023

4.4 CU-04 — Escalamiento automático por SLA

EP-CU04-01 — SLA vencido eleva ticket de L1 a L2

Campo	Valor
Título descriptivo	Un Incidente Crítico sin actividad supera los 15 minutos del SLA; el job de escalamiento lo eleva a nivel L2 y notifica al agente y al admin.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. <code>TKT-0080</code> Incidente Crítico, creado hace 16 minutos (o reloj manipulado con feature flag de test), sin comentarios, sin cambios de estado, <code>nivel=L1</code> . 3. <code>agente1</code> asignado. 4. SLA configurado: Incidente Crítico = 15 min. 5. EventBridge Scheduler con el job <code>sla-escalation-job</code> habilitado.
Data	Trigger manual del job vía <code>aws scheduler invoke</code> o esperar la siguiente corrida (cada 5 min).
Pasos a ejecutar	1. Verificar precondición del tiempo (<code>GET /tickets/TKT-0080</code> y comparar <code>createdAt</code> vs <code>now</code>). 2. Disparar el job (manual o esperar ciclo). 3. <code>GET /tickets/TKT-0080</code> . 4. <code>GET /tickets/TKT-0080/history</code> . 5. Revisar Mailpit del agente y del admin.
Resultado esperado	• Paso 3 → <code>nivel=L2</code> . • Paso 4 → último evento <code>ESCALATED</code> con <code>from=L1</code> , <code>to=L2</code> , <code>reason=SLA_EXPIRED</code> , <code>slaConfiguredMin=15</code> , <code>elapsedMin>=16</code> , <code>actor=system</code> . • Paso 5 → email al agente y al admin con subject <code>[TKT-0080] Escalado a L2 por SLA</code> .
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Re-crear un ticket con timestamp ajustado para el próximo run (no es posible retroceder el estado de escalamiento). 2. Drenar Mailpit. 3. Registrar en Qase.io.
Adjuntos	<code>docs/mockups/07_escalamiento_sla.png</code> .
Tipo	Camino feliz
Prioridad	Crítica
Relacionado con (E2)	<code>TI-103</code> , <code>TUAT-004</code> , <code>TE2E-004</code> , <code>TU-004</code> , <code>TU-005</code>

EP-CU04-02 — Ticket con actividad reciente NO escala

Campo	Valor
Título descriptivo	Un Incidente Crítico con un comentario hace 5 minutos no es escalado cuando se ejecuta el job; permanece en L1 .
Precondiciones	1. Ambiente ENT-004 staging . 2. TKT-0081 Incidente Crítico, creado hace 16 min, nivel=L1 . 3. Existe un comentario sobre el ticket creado hace 5 min (timestamp del último comentario < SLA configurado). 4. Job sla-escalation-job listo para disparo manual.
Data	Trigger manual del job.
Pasos a ejecutar	1. Verificar lastActivityAt del ticket. 2. Disparar el job. 3. GET /tickets/TKT-0081 . 4. GET /tickets/TKT-0081/history y verificar que NO hay evento ESCALATED posterior al disparo.
Resultado esperado	• Paso 3 → nivel=L1 (sin cambios). • Paso 4 → último evento sigue siendo el comentario; no aparece ESCALATED . • Logs del job muestran a TKT-0081 con razón SKIPPED: recent activity .
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io. 2. Mantener el ticket para el próximo run sin nuevos comentarios.
Adjuntos	(sin adjuntos)
Tipo	Negativo
Prioridad	Alta
Relacionado con (E2)	TI-104

EP-CU04-03 — Escalamiento máximo se detiene en L3

Campo	Valor
Título descriptivo	Un ticket que ya alcanzó L3 no es escalado a un nivel mayor cuando el job se ejecuta de nuevo; el evento <code>ESCALATION_CAP_REACHED</code> queda en el historial.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. <code>TKT-0082</code> Incidente Crítico, <code>nivel=L3</code> , sin actividad reciente. 3. Job listo para disparo manual.
Data	Trigger manual del job.
Pasos a ejecutar	1. Disparar el job. 2. <code>GET /tickets/TKT-0082</code> . 3. <code>GET /tickets/TKT-0082/history</code> .
Resultado esperado	• Paso 2 → <code>nivel=L3</code> (no <code>L4</code>). • Paso 3 → un evento <code>ESCALATION_CAP_REACHED</code> con <code>nivelActual=L3</code>, <code>escalamientoMaximoAlcanzado=true</code>. • Logs del job muestran <code>TKT-0082</code> con razón <code>SKIPPED: max level reached</code>. • Notificación al admin con subject <code>[TKT-0082] Escalamiento máximo alcanzado – requiere intervención</code>.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Drenar Mailpit. 2. Registrar en Qase.io.
Adjuntos	<code>docs/mockups/07_escalamiento_sla.png</code> .
Tipo	Borde
Prioridad	Alta
Relacionado con (E2)	<code>TU-006</code>

4.5 CU-05 — Historial inmutable

EP-CU05-01 — Timeline ordenado cronológicamente

Campo	Valor
Título descriptivo	Un ticket con 5 eventos (creación, asignación, dos comentarios, resolución) devuelve un historial ordenado por timestamp ascendente con autor y descripción.
Precondiciones	1. Ambiente ENT-004 staging . 2. TKT-0090 con exactamente 5 eventos sembrados: TICKET_CREATED (T0), TICKET_ASSIGNED (T0+2m), COMMENT_ADDED (T0+5m), COMMENT_ADDED (T0+10m), RESOLVED (T0+45m). 3. Usuario agente1 autenticado.
Data	JWT de agente1.
Pasos a ejecutar	1. GET /tickets/TKT-0090/history . 2. Verificar el orden, los autores y los timestamps. 3. Verificar formato UTC (Z al final).
Resultado esperado	• 200 OK . • Array de exactamente 5 eventos. • events[0].type=TICKET_CREATED , events[4].type=RESOLVED . • events[i].timestamp <= events[i+1].timestamp para todo i. • Cada evento incluye autor , timestamp , description .
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io.
Adjuntos	docs/mockups/06_historial.png .
Tipo	Camino feliz
Prioridad	Alta
Relacionado con (E2)	TI-026 , TUAT-005 , TE2E-005 , TU-018

EP-CU05-02 — UPDATE directo sobre evento es bloqueado

Campo	Valor
Título descriptivo	Un intento de modificar un evento existente del historial (vía endpoint hipotético o vía SQL directo en <code>ticket_events</code>) es rechazado por la regla de inmutabilidad.
Precondiciones	1. Ambiente <code>ENT-003 dev shared</code> o <code>ENT-004 staging</code> . 2. <code>TKT-0091</code> con al menos 1 evento existente. 3. Acceso administrativo a RDS (usuario <code>tester_admin</code> con permisos <code>UPDATE</code> sobre <code>ticket_events</code>).
Data	SQL: <code>UPDATE ticket_events SET description='alterado' WHERE id=<eventoId>;</code> Y, vía API, <code>PUT /events/<eventoId></code> con body arbitrario.
Pasos a ejecutar	1. Intentar el <code>PUT /events/<eventoId></code> (vía cliente HTTP). 2. Intentar el <code>UPDATE</code> SQL directo en la base. 3. <code>GET /tickets/TKT-0091/history</code> y verificar que el contenido no cambió.
Resultado esperado	• Paso 1 → <code>405 Method Not Allowed</code> o <code>404 Not Found</code> (no existe endpoint de modificación). • Paso 2 → error del trigger <code>prevent_update_ticket_events</code> (o RLS): <code>ImmutableEventError</code> . • Paso 3 → eventos intactos, descripciones originales.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Revisar logs de auditoría (CloudTrail / pgAudit) para confirmar que el intento quedó registrado. 2. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Negativo / seguridad
Prioridad	Crítica
Relacionado con (E2)	<code>TU-015</code> , <code>TSEC</code> (sección RBAC + inmutabilidad)

EP-CU05-03 — Adjuntos del historial accesibles vía URL prefirmada

Campo	Valor
Título descriptivo	Cada evento del historial que referencia un adjunto en S3 devuelve una URL prefirmada con expiración corta; la URL deja de funcionar después del TTL.
Precondiciones	1. Ambiente ENT-004 staging . 2. TKT-0092 con un evento que referencia tickets/2026/05/adjunto.png en S3. 3. Usuario autorizado (autor del ticket o agente).
Data	JWT del usuario. TTL configurado para URLs prefirmadas: 300 s (5 min).
Pasos a ejecutar	1. GET /tickets/TKT-0092/history . 2. Extraer la URL prefirmada del evento con adjunto. 3. GET a la URL prefirmada (sin auth) y verificar que devuelve el archivo. 4. Esperar 310 s (TTL + margen). 5. Repetir GET a la misma URL.
Resultado esperado	• Paso 3 → 200 OK ; el body es el binario del archivo; Content-Type correcto. • La URL contiene X-Amz-Expires=300 . • Paso 5 → 403 Forbidden de S3 con mensaje de firma expirada.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io (anotando que el escenario consume ~6 min reales).
Adjuntos	(sin adjuntos)
Tipo	Borde
Prioridad	Alta
Relacionado con (E2)	TI-080 , TI-081 , TU-010 , TU-011

4.6 CU-06 — Filtrado de la cola

EP-CU06-01 — Filtros combinados estado + prioridad + agente

Campo	Valor
Título descriptivo	El endpoint <code>GET /tickets</code> con tres filtros combinados devuelve únicamente los tickets que cumplen las tres condiciones, paginados correctamente.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> con dataset semilla de 200 tickets variados (mezcla de estados, prioridades, assignees). 2. Usuario con rol Agente o Admin autenticado.
Data	Query: <code>?estado=EnProgreso&prioridad=Alta&assignee=agente1@galileo.gt&limit=20</code> .
Pasos a ejecutar	1. <code>GET /tickets</code> con la query del campo Data. 2. Para cada item del array, verificar que cumple las 3 condiciones. 3. Validar que <code>limit=20</code> se respeta. 4. Si <code>nextCursor</code> está presente, hacer la segunda página y revalidar.
Resultado esperado	• <code>200 OK</code>. • Todos los items tienen <code>estado=EnProgreso</code>, <code>prioridad=Alta</code> y <code>assignee=agente1</code>. • <code>items.length <= 20</code>. • <code>nextCursor</code> ausente si hay menos de 20 items en total.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io.
Adjuntos	<code>docs/mockups/02_cola_tickets.png</code> .
Tipo	Camino feliz
Prioridad	Alta
Relacionado con (E2)	<code>TI-060</code> , <code>TI-061</code> , <code>TI-062</code> , <code>TUAT-006</code> , <code>TE2E-006</code>

EP-CU06-02 — Reportante solo ve sus tickets (RBAC)

Campo	Valor
Título descriptivo	Un reportante con 3 tickets propios consulta la cola sin filtros y recibe solo sus tickets, no los de otros usuarios.
Precondiciones	1. Ambiente ENT-004 staging . 2. reportante3@galileo.gt con exactamente 3 tickets propios (TKT-0100 , TKT-0101 , TKT-0102). 3. Dataset general con 100 tickets ajenos.
Data	JWT de reportante3.
Pasos a ejecutar	1. GET /tickets sin filtros (con JWT del reportante). 2. Conteo del array. 3. Intentar GET /tickets/<TKT-de-otro> con un id ajeno.
Resultado esperado	• Paso 1 → 200 OK ; exactamente 3 items; todos con reporterId=reportante3 . • Paso 3 → 403 Forbidden o 404 Not Found (consistente con la política definida para IDOR en E1 §4.7).
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io. 2. Confirmar evento de denegación en CloudWatch.
Adjuntos	(sin adjuntos)
Tipo	Error controlado / RBAC
Prioridad	Crítica
Relacionado con (E2)	TI-064 , TU-009 , TSEC-002 (IDOR)

EP-CU06-03 — Latencia P95 < 2 s con 10k tickets

Campo	Valor
Título descriptivo	Bajo 10k tickets sembrados y 100 consultas concurrentes con filtros, el P95 de latencia del endpoint <code>GET /tickets</code> se mantiene por debajo de 2000 ms.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> con dataset de 10 000 tickets generado vía Faker. 2. Índices recomendados creados en <code>tickets(estado, prioridad)</code> y <code>tickets(assignee_id)</code> . 3. k6 instalado en la máquina del tester (versión ≥ 0.50).
Data	Script k6 que genera 100 VUs durante 60 s con query: <code>?estado=Abierto&prioridad=Alta</code> .
Pasos a ejecutar	1. Ejecutar el script k6: <code>k6 run scripts/load/cu06.js</code> . 2. Recoger el reporte HTML/JSON. 3. Leer la métrica <code>http_req_duration{expected_response:true}</code> percentil P95.
Resultado esperado	• k6 exit code 0. • P95 < 2000 ms (criterio CU-06 §3 E1). • Tasa de error HTTP < 1%. • RDS sin warnings de slow query.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Archivar el reporte k6 en <code>docs/evidencias/EP-CU06-03-k6.html</code> . 2. Registrar en Qase.io adjuntando el reporte. 3. Drenar conexiones colgadas si fuera el caso.
Adjuntos	<code>scripts/load/cu06.js</code> (script k6), <code>docs/evidencias/EP-CU06-03-k6.html</code> (al ejecutar).
Tipo	Borde / rendimiento
Prioridad	Crítica
Relacionado con (E2)	<code>TI-063</code> , <code>TC-001</code> (sección §9 Pruebas de carga)

4.7 CU-07 — Reporte por período

EP-CU07-01 — Administrador descarga reporte mensual en CSV

Campo	Valor
Título descriptivo	Un administrador descarga el reporte agregado del mes de mayo de 2026 en formato CSV con totales por categoría y por agente.
Precondiciones	1. Ambiente ENT-004 staging con dataset semilla del mes mayo/2026 (al menos 50 tickets resueltos). 2. Usuario admin1 autenticado. 3. Reglas de cálculo del reporte deployadas (suma, promedio, distribución).
Data	Query: ?from=2026-05-01&to=2026-05-31&format=csv .
Pasos a ejecutar	1. GET /reports?from=2026-05-01&to=2026-05-31&format=csv con JWT del admin. 2. Verificar headers de respuesta. 3. Descargar el CSV. 4. Validar manualmente las columnas y un total contra un script de verificación independiente.
Resultado esperado	• 200 OK . • Content-Type: text/csv . • Content-Disposition: attachment; filename="reporte_2026-05.csv" . • CSV con columnas: agente , categoria , total_resueltos , tiempo_promedio_resolucion_min , cumplimiento_sla_pct . • Los totales coinciden con la query SQL de verificación.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Archivar el CSV descargado en docs/evidencias/EP-CU07-01-reporte.csv . 2. Registrar en Qase.io.
Adjuntos	docs/mockups/08_reportes.png , docs/evidencias/EP-CU07-01-reporte.csv (al ejecutar).
Tipo	Camino feliz
Prioridad	Media
Relacionado con (E2)	TUAT-007 , TE2E-007 , TU-017

EP-CU07-02 — Agente NO puede descargar reporte global (RBAC)

Campo	Valor
Título descriptivo	Un agente autenticado recibe <code>403 Forbidden</code> al intentar descargar el reporte global; ningún CSV se entrega.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. Usuario <code>agente1</code> autenticado.
Data	JWT de agente1. Query: <code>?from=2026-05-01&to=2026-05-31&format=csv</code> .
Pasos a ejecutar	1. <code>GET /reports?...</code> con el JWT del agente. 2. Leer el código y body.
Resultado esperado	• <code>403 Forbidden</code>. • Body con <code>ROLE_NOT_ALLOWED</code>, <code>role=Agente</code>, <code>action=download_global_report</code>. • Ningún CSV en el body. • Evento de denegación en CloudWatch.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Error controlado / RBAC
Prioridad	Crítica
Relacionado con (E2)	TSEC (RBAC matrix), por extensión de <code>TI-024</code> y <code>TU-007</code>

EP-CU07-03 — Reporte con rango vacío devuelve CSV con headers

Campo	Valor
Título descriptivo	Un reporte solicitado para un rango sin tickets resueltos devuelve un CSV bien formado con solo la fila de headers; no error 404.
Precondiciones	1. Ambiente ENT-004 staging con un rango histórico sin tickets (ej. enero/2025 antes del go-live). 2. Usuario admin1 autenticado.
Data	Query: ?from=2025-01-01&to=2025-01-31&format=csv .
Pasos a ejecutar	1. GET /reports?... . 2. Descargar y abrir el CSV. 3. Contar filas.
Resultado esperado	• 200 OK (no 404). • CSV con exactamente 1 línea (headers de columnas) y 0 filas de datos. • Headers idénticos a los del reporte poblado (EP-CU07-01).
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Borde
Prioridad	Baja
Relacionado con (E2)	Extensión de TUAT-007

4.8 CU-08 — Solicitud de servicio en cola separada

EP-CU08-01 — Solicitud aparece solo en cola de solicitudes

Campo	Valor
Título descriptivo	Un ticket con <code>tipo=Solicitud</code> aparece únicamente en la cola de solicitudes; no contamina la cola de incidentes.
Precondiciones	1. Ambiente <code>ENT-004 staging</code> . 2. Reportante autenticado.
Data	Cuerpo: <code>{ "tipo": "Solicitud", "severidad": "Media", "titulo": "Acceso a repositorio interno", "descripcion": "Requiero permisos de lectura al repo X" }</code> .
Pasos a ejecutar	1. <code>POST /tickets</code> con el cuerpo Data; capturar <code>id=TKT-N</code> . 2. <code>GET /tickets?tipo=Solicitud</code> y verificar que <code>TKT-N</code> aparece. 3. <code>GET /tickets?tipo=Incidente</code> y verificar que <code>TKT-N</code> NO aparece.
Resultado esperado	• Paso 1 → <code>201 Created</code> ; body con <code>tipo=Solicitud</code> . • Paso 2 → array contiene <code>TKT-N</code> . • Paso 3 → array NO contiene <code>TKT-N</code> .
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Limpiar el ticket en el próximo re-seed. 2. Registrar en Qase.io.
Adjuntos	<code>docs/mockups/04_crear_solicitud.png</code> .
Tipo	Camino feliz
Prioridad	Media
Relacionado con (E2)	<code>TI-002</code> , <code>TUAT-008</code> , <code>TE2E-008</code>

EP-CU08-02 — Tipo inválido devuelve 422

Campo	Valor
Título descriptivo	Un POST de creación con tipo fuera del enum {Incidente, Solicitud} es rechazado con 422 Unprocessable Entity .
Precondiciones	1. Ambiente ENT-004 staging . 2. Reportante autenticado.
Data	Cuerpo: { "tipo": "Otro", "severidad": "Media", "titulo": "X", "descripcion": "Y" } .
Pasos a ejecutar	1. POST /tickets con el cuerpo Data. 2. Leer el código y body.
Resultado esperado	• 422 Unprocessable Entity . • Body con código INVALID_TYPE , allowedValues= ["Incidente","Solicitud"] . • Ningún registro persistido.
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io.
Adjuntos	(sin adjuntos)
Tipo	Error controlado
Prioridad	Alta
Relacionado con (E2)	TU-003 , TU-014

EP-CU08-03 — Solicitud de baja severidad no dispara escalamiento

Campo	Valor
Título descriptivo	Una Solicitud con severidad=Baja creada hace 24 horas sin actividad no es escalada por el job de SLA (el SLA aplica solo a incidentes).
Precondiciones	1. Ambiente ENT-004 staging . 2. TKT-0200 Solicitud Baja, creada hace 24 h, sin actividad, nivel=L1 . 3. Job sla-escalation-job listo para disparo.
Data	Trigger manual del job.
Pasos a ejecutar	1. Disparar el job. 2. GET /tickets/TKT-0200 . 3. GET /tickets/TKT-0200/history .
Resultado esperado	• Paso 2 → nivel=L1 sin cambios. • Paso 3 → ningún evento ESCALATED posterior al disparo. • Logs del job muestran TKT-0200 con razón SKIPPED: type=Solicitud (no SLA) .
Estado de ejecución	No ejecutado
Acciones post ejecución	1. Registrar en Qase.io. 2. Mantener el ticket para próximos runs.
Adjuntos	(sin adjuntos)
Tipo	Borde
Prioridad	Media
Relacionado con (E2)	Extensión funcional de TI-104 para tipo Solicitud

5. Matriz consolidada de escenarios

ID	CU	Tipo	Prioridad	Estado	Referencia E2 principal
EP-CU01-01	CU-01	Feliz	Crítica	No ejecutado	TI-001, TUAT-001
EP-CU01-02	CU-01	Error	Alta	No ejecutado	TI-003
EP-CU01-03	CU-01	Borde	Alta	No ejecutado	TI-007
EP-CU02-01	CU-02	Feliz	Crítica	No ejecutado	TI-040, TUAT-002
EP-CU02-02	CU-02	RBAC	Crítica	No ejecutado	TI-024, TU-007
EP-CU02-03	CU-02	Borde	Media	No ejecutado	TI-043
EP-CU03-01	CU-03	Feliz	Crítica	No ejecutado	TI-022, TUAT-003
EP-CU03-02	CU-03	Error	Crítica	No ejecutado	TI-021, TU-016
EP-CU03-03	CU-03	Borde	Alta	No ejecutado	TI-023
EP-CU04-01	CU-04	Feliz	Crítica	No ejecutado	TI-103, TUAT-004
EP-CU04-02	CU-04	Negativo	Alta	No ejecutado	TI-104
EP-CU04-03	CU-04	Borde	Alta	No ejecutado	TU-006
EP-CU05-01	CU-05	Feliz	Alta	No ejecutado	TUAT-005, TI-026
EP-CU05-02	CU-05	Negativo	Crítica	No ejecutado	TU-015
EP-CU05-03	CU-05	Borde	Alta	No ejecutado	TI-081, TU-010
EP-CU06-01	CU-06	Feliz	Alta	No ejecutado	TI-061, TUAT-006
EP-CU06-02	CU-06	RBAC	Crítica	No ejecutado	TI-064, TU-009
EP-CU06-03	CU-06	Rendimiento	Crítica	No ejecutado	TI-063, TC-001
EP-CU07-01	CU-07	Feliz	Media	No ejecutado	TUAT-007
EP-CU07-02	CU-07	RBAC	Crítica	No ejecutado	TI-024 (extensión)
EP-CU07-03	CU-07	Borde	Baja	No ejecutado	TUAT-007 (extensión)
EP-CU08-01	CU-08	Feliz	Media	No ejecutado	TI-002, TUAT-008
EP-CU08-02	CU-08	Error	Alta	No ejecutado	TU-003, TU-014
EP-CU08-03	CU-08	Borde	Media	No ejecutado	TI-104 (extensión)

Totales: 24 escenarios = 8 felices + 4 errores controlados + 5 RBAC/negativos + 6 bordes + 1 rendimiento.

Anexo IA — Uso responsable de asistencia con IA

Qué le pedimos a la IA

- Ayudar a derivar 3 escenarios por cada CU (camino feliz, error controlado, borde) a partir de los criterios de éxito de E1 y los casos `TI-` / `TUAT-` ya existentes en E2.
- Sugerir la **estructura de tabla vertical** (Campo / Valor) como mejor forma de respetar los 8 campos del slide manteniendo legibilidad para pasos multi-línea.
- Producir la comparativa de herramientas (Qase, TestRail, TestLink, GitHub Issues, Notion) contra los criterios de capa gratuita, cobertura de campos, export y trazabilidad.
- Generar el CSV de import a Qase con el mapeo de columnas requerido por su importer.

Qué aceptamos y editamos

- La elección de **Qase.io free tier** como herramienta principal: se aceptó tras verificar manualmente que el plan free es permanente (no trial).
- La numeración `EP-CUNN-XX` para evitar colisión con los prefijos de E2 (`TU-`, `TI-`, `TUAT-`, etc.).
- La separación deliberada entre este documento (escenarios funcionales por CU) y E2 (estrategia y pruebas no funcionales): la IA propuso fusionar todo en un solo documento; el equipo decidió mantenerlos separados para evitar romper el contrato de inmutabilidad de E2.
- El campo "Acciones post ejecución" incluye explícitamente **limpieza de datos** (drenar Mailpit, marcar tickets para re-seed, archivar evidencias), lo cual la IA inicialmente había omitido.

Qué descartamos y por qué

- **Más de 3 escenarios por CU:** la IA sugirió ~6 por CU (incluyendo seguridad y carga); se descartó porque esas dimensiones ya están cubiertas en E2 §8-§11 y duplicarlas rompe DRY.
- **Adjuntar capturas de Qase.io en este PDF:** se descartó porque requiere ejecutar la herramienta antes del cierre de esta entrega; se opta por incluir el CSV (`docs/E3_qase_import.csv`) que permite reproducir el registro en cualquier momento.
- **Generar el PDF con MarkdownPDF VSCode extension** (el método inferido de E2): no se descarta, pero se opta por replicar el flujo con `pandoc → HTML → Google Chrome headless` para que sea reproducible desde la línea de comandos sin depender del editor.
- **Usar Notion como respaldo de Qase:** la IA lo propuso; se descartó por evitar dispersión de la evidencia entre múltiples herramientas (el profesor solo necesita ver una).

Fin del documento — Entrega #3 · Escenarios de Prueba.